

Play-O-Meter Infrastructure

Infrastructure Stack

Here is a recommended infrastructure stack:

Component	Technology	Purpose
PC App	Electron	Compile to a native executable; handles authentication pings, local session recording and file management.
Authentication	API Keys / Client Credentials, OAuth 2.0, mTLS custom JWT	Manage user logins, issues tokens, or handles private key/certificate validation.
Backend API	Python (FastAPI), or Go	Validates handshake, generates secure upload links, and serves web app data.
Database	PostgreSQL, MongoDB for users + metadata. Lakes for events and logs.	Store session metadata (timestamps, user IDs, link to the stored video file, etc..).
Cloud/Server Storage	Blob Storage from a cloud provider or S3 Wrapper for local server	Stores the heavy video files.

Web App	React, Vue, or Next.js	Dashboard for users to view their session data and stream the recordings.
----------------	------------------------	---

Waterfall Deployment

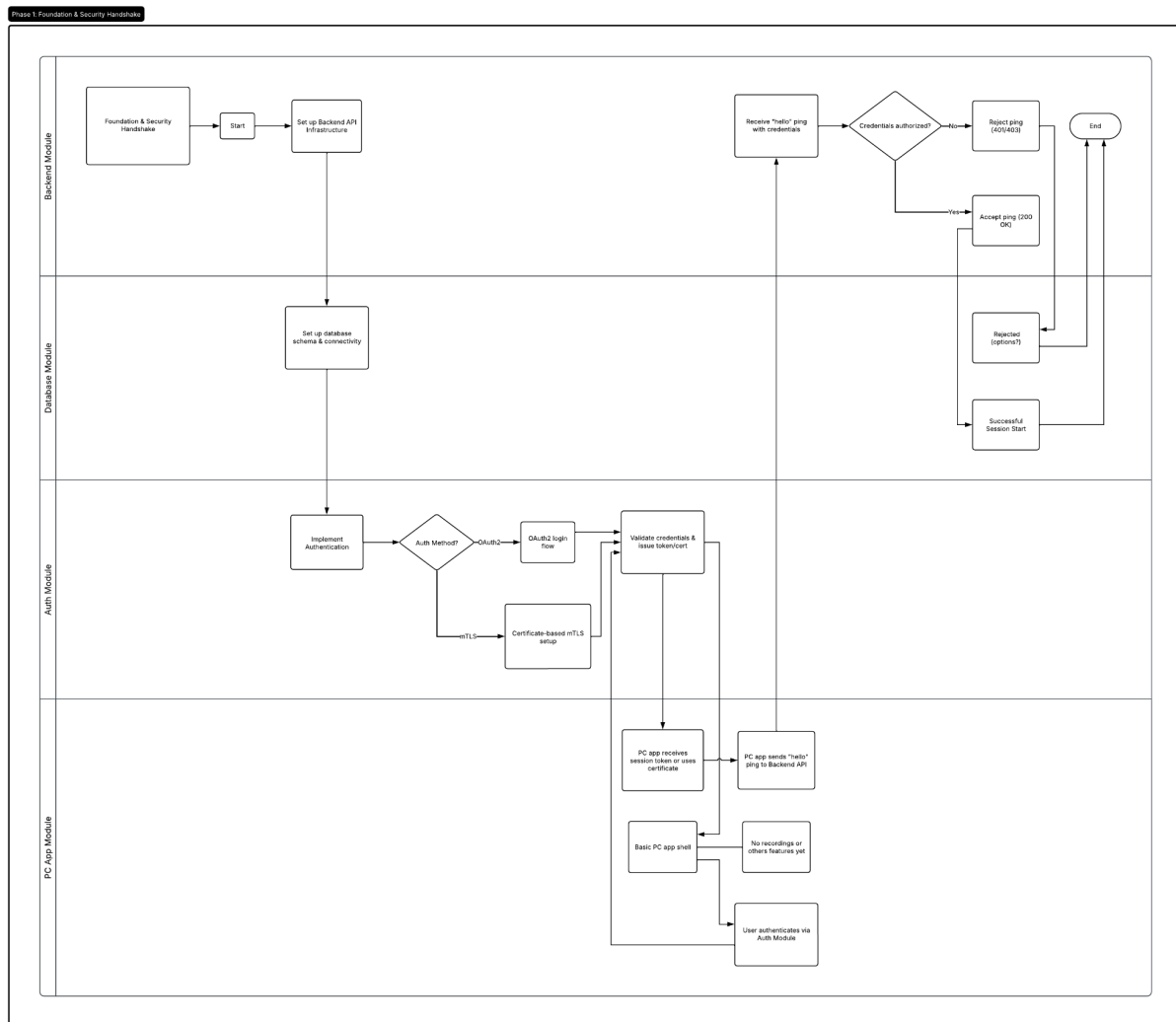
In a waterfall model, each phase must be completed and validated before moving to the next.
No cut corners

Phase 1: Foundation & Security Handshake

Establish a secure, authenticated connection between a compiled PC app and our server.

- Set up the Backend API and Database infrastructure.
- Implement the authentication provider.
- Compile the basic PC app shell (no recording features yet).
- Program the PC app to authenticate the user and retrieve a session token or pass a secure certificate.
- Program the PC app to send a "hello" ping to the Backend API using the credentials.
- Verify the API rejects unauthorized pings and accepts authorized ones, logging the successful session start in the database.
- Uther tip: Implement error handling for the PC app from the get go (e.g., what happens if the internet drops during the video upload, or what happens if the user keeps logging-in and out, can I start and close sessions 100 times in 10 seconds ?).

Diagram



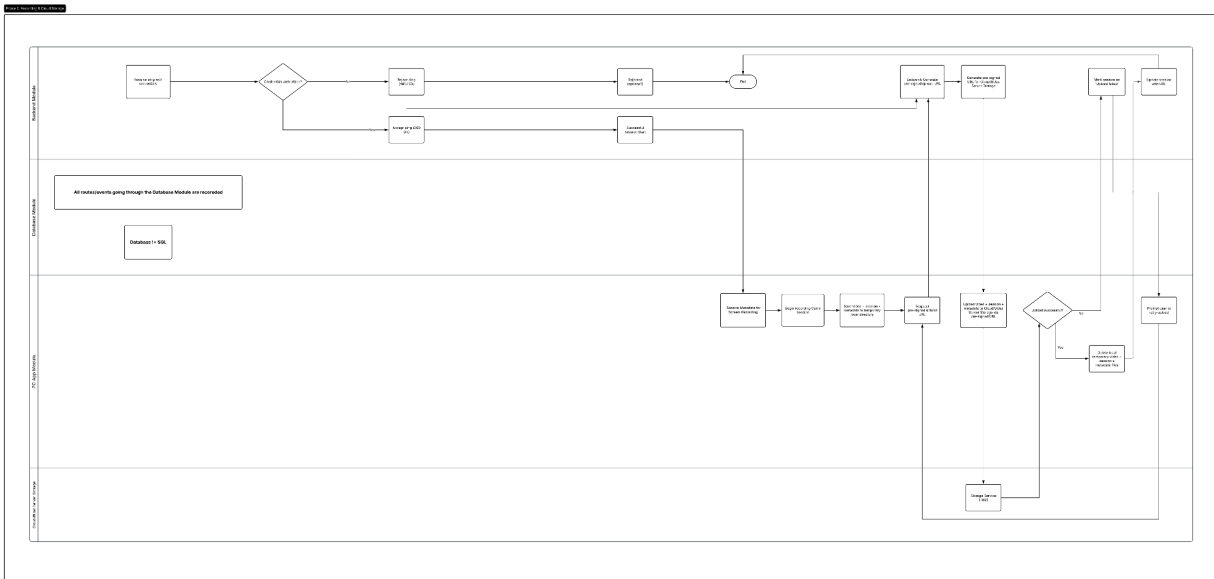
Phase 2: Local Recording & Cloud Storage

Once the API trusts the PC app, we start to capture sessions.

- Configure the app to begin recording only *after* the Phase 1 authentication handshake is green.
- Set the app to save the recording files locally to a temporary, hidden directory on the user's machine.
- Create an endpoint on the Backend API that generates a "Pre-signed URL" (a temporary, secure direct-upload link to your Cloud Storage).
- Program the PC app to request this URL, upload the local files directly to the Cloud Storage, and delete the local temporary files to free up user disk space.
- Update the Database session record with the cloud storage URL of the recording.

- Uther tip: Make sure the data makes sense and that your model is really capturing all the important points when it comes to logging user interactions.

Diagram

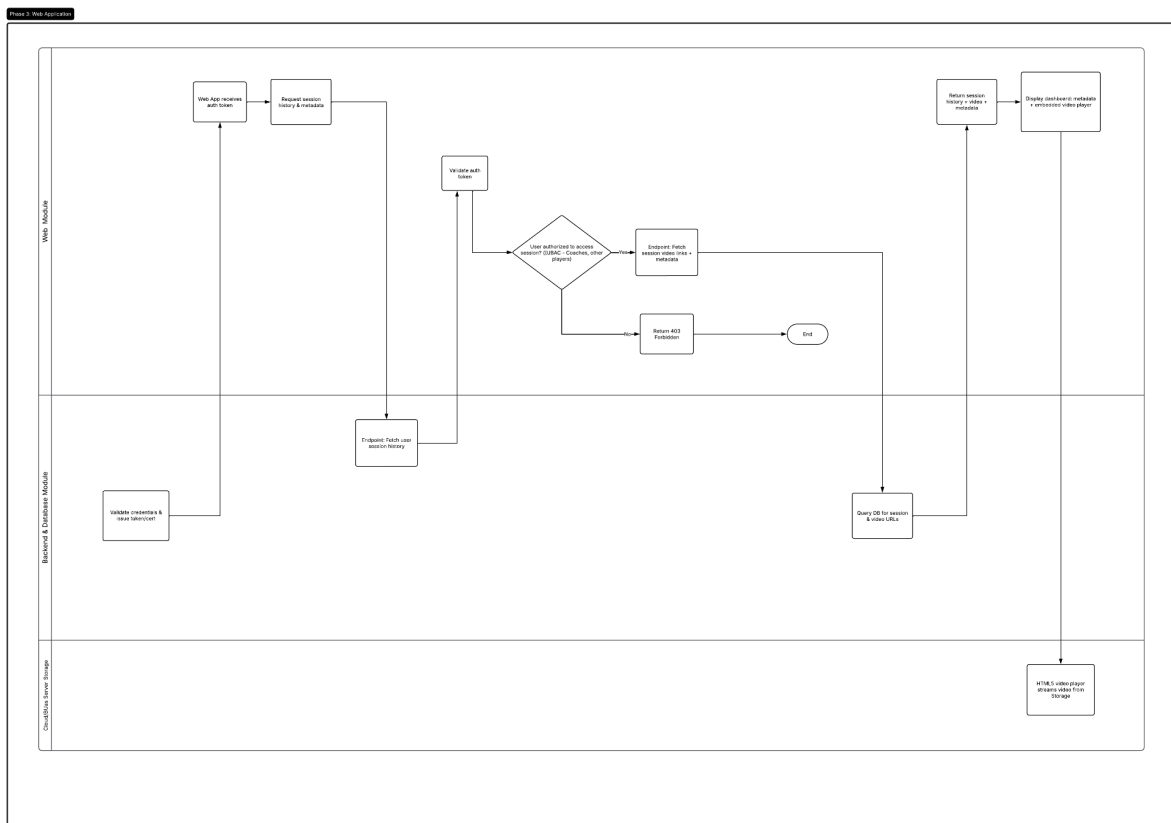


Phase 3: Web Application & Data Synchronization

Now that data is flowing from the PC to the cloud, we need a well made-web app.

- Deploy the Web App frontend (That step will be more detailed as we get on with the project).
- Build a login for the Web App so users can only access their own data.
- Create Backend API endpoints to fetch a user's session history and video links.
- Implement a player in the Web App to stream the session directly from the Cloud Storage (video, keystrokes and all).
- Uther tip : Design the dashboard UI to look good, really take time to study UX/UI design.

Diagram



Phase 4: End-to-End Testing & Polish

The entire flow must be tested as a single unit.

- Testing the API to ensure no one can upload or view videos without a valid token.
- Test the PC app's resource usage during screen recording
- Finalize the installer and code-sign the application so Windows does not flag it as malware.